

Script Explanation Request



please explain me the script of arvr i will give you
please try too keep it short and simple but tell all

Sure—send the script.

I'll explain it in a **short, simple, viva-friendly way**, covering:

1. **What the script does** (overall purpose)
2. **Each variable** (why it is used)
3. **Each function** (what happens step by step)
4. **Important Unity concepts used** (like Input System, Light, GameObject, etc.)
5. **1-2 line viva answer** you can say in exam

Paste the code whenever you're ready.



```
using UnityEngine;
using UnityEngine.InputSystem;

public class DayNightToggle : MonoBehaviour
{
    [Header("Light Reference")]
    public Light directionalLight;

    [Header("Day Settings")]
    public Color dayLightColor = Color.white;
    public float dayIntensity = 1.2f;
    public Color daySkyColor = new Color(0.53f, 0.81f, 0.98f);

    [Header("Night Settings")]
    public Color nightLightColor = new Color(0.1f, 0.1f, 0.35f);
    public float nightIntensity = 0.3f;
    public Color nightSkyColor = new Color(0.02f, 0.02f, 0.05f);

    private bool isDay = true;

    void Start()
    {
```

```

    SetDay();
}

void Update()
{
    // NEW INPUT SYSTEM: Mouse click
    if (Mouse.current.leftButton.wasPressedThisFrame)
    {
        if (isDay)
            SetNight();
        else
            SetDay();
    }
}

void SetDay()
{
    RenderSettings.ambientLight = daySkyColor;
    directionalLight.color = dayLightColor;
    directionalLight.intensity = dayIntensity;
    isDay = true;
}

void SetNight()
{
    RenderSettings.ambientLight = nightSkyColor;
    directionalLight.color = nightLightColor;
    directionalLight.intensity = nightIntensity;
    isDay = false;
}
}

```

Script Name: DayNightToggle

Purpose:

This script changes the Unity scene from **Day** → **Night** and **Night** → **Day** when the **left mouse button** is clicked.

1) Libraries Used

```

C#
using UnityEngine;
using UnityEngine.InputSystem;

```

```
using UnityEngine;
```

Used for Unity basic features like:

- Light

- Color
- MonoBehaviour
- RenderSettings

using **UnityEngine.InputSystem**;

Used for **New Input System** (mouse/keyboard/controller input).

Here we use it to detect **mouse click**.

2) Class Creation

C#
public class *DayNightToggle* : *MonoBehaviour*

- Creates a script named **DayNightToggle**
 - **MonoBehaviour** means this script can be attached to a **GameObject** in Unity
 - Unity automatically calls functions like:
 - **Start()**
 - **Update()**
-

3) Light Reference

C#
[Header("Light Reference")]
public *Light* *directionalLight*;

Creates a variable to store the **Directional Light (Sun)**.

Why?

To change:

- light color
- brightness

Example:

- Day = bright white sunlight
- Night = dim blue moonlight

In Unity Inspector, you drag your **Directional Light** here.

4) Day Settings

```
C#
public Color dayLightColor = Color.white;
```

Day light color = **White**

```
C#
public float dayIntensity = 1.2f;
```

Brightness = **1.2** (bright)

```
C#
public Color daySkyColor = new Color(0.53f, 0.81f, 0.98f);
```

Sky/ambient color = **light blue**

Makes scene look daytime.

5) Night Settings

```
C#
public Color nightLightColor = new Color(0.1f, 0.1f, 0.35f);
```

Dark blue light → moonlight effect

```
C#
public float nightIntensity = 0.3f;
```

Low brightness → dark scene

```
C#
public Color nightSkyColor = new Color(0.02f, 0.02f, 0.05f);
```

Almost black sky → night effect

6) Boolean Variable

```
C#
private bool isDay = true;
```

Boolean stores:

- **true**
- **false**

Here:

- **true** = Day
- **false** = Night

Initially:

```
C#
isDay = true
```

Scene starts in **Day mode**

7) Start Function

```
C#
void Start()
{
    SetDay();
}
```

Start() runs **once** when game starts.

Calls:

```
C#
SetDay();
```

So scene begins as **Day**

8) Update Function

```
C#
void Update()
```

Runs every frame continuously.

Checks for input.

```
C#
if (Mouse.current.LeftButton.wasPressedThisFrame)
```

Meaning:

If left mouse button is clicked once

then run code below.

9) Toggle Logic

```
C#
if (isDay)
    SetNight();
else
    SetDay();
```

If currently day:

→ make night

Otherwise:

→ make day

Example:

Click 1:

Day → Night

Click 2:

Night → Day

Click 3:

Day → Night

Keeps switching.

10) SetDay Function

```
C#  
void SetDay()
```

Used to make scene daytime.

Ambient Light

```
C#  
RenderSettings.ambientLight = daySkyColor;
```

Changes environment light to blue sky color.

Directional Light Color

```
C#  
directionalLight.color = dayLightColor;
```

Makes sunlight white.

Brightness

```
C#  
directionalLight.intensity = dayIntensity;
```

Bright sunlight.

Update state

```
C#  
isDay = true;
```

Now program knows it is daytime.

11) SetNight Function

C#
void *SetNight()*

Used to make scene nighttime.

Ambient Light

C#
RenderSettings.ambientLight = *nightSkyColor*;

Dark environment.

Light Color

C#
directionalLight.color = *nightLightColor*;

Blue moonlight effect.

Brightness

C#
directionalLight.intensity = *nightIntensity*;

Low brightness.

Update state

C#
isDay = **false**;

Now scene is night.

Flow of Script

Game starts

↓

Start() runs

↓

SetDay()

↓

Day scene shown

↓

Mouse click

↓

Update() detects click

↓
SetNight()
 ↓
 Night scene shown
 ↓
 Mouse click again
 ↓
SetDay()

Viva Answer (short)

“This script creates a Day/Night toggle system in Unity. It uses the New Input System to detect left mouse click. On click, it changes ambient light, directional light color, and intensity to switch between day and night scene.”



```

☐
using UnityEngine;
using UnityEngine.InputSystem;

public class CandleToggle : MonoBehaviour
{
    public GameObject flame;
    public GameObject smoke;
    public Light candleLight;

    private bool isLit = false;

    void Update()
    {
        // Check left mouse click using New Input System
        if (Mouse.current != null && Mouse.current.leftButton.wasPressedThisFrame)
        {
            isLit = !isLit;

            flame.SetActive(isLit);
            smoke.SetActive(isLit);
            candleLight.enabled = isLit;
        }
    }
}

```

Script Name: CandleToggle

Purpose:

This script **turns a candle ON/OFF on mouse click.**

When clicked:

- Flame appears 🔥
- Smoke appears 🌫️
- Light turns ON 💡

Click again:

- Everything turns OFF.

1) Libraries

C#

```
using UnityEngine;
using UnityEngine.InputSystem;
```

UnityEngine

Used for:

- GameObject
- Light
- MonoBehaviour

UnityEngine.InputSystem

Used for **New Input System** mouse input.

2) Class

C#

```
public class CandleToggle : MonoBehaviour
```

Creates script named **CandleToggle**.

MonoBehaviour allows Unity to run functions like:

- **Start()**
- **Update()**

Attach this script to any GameObject (like Candle).

3) Variables

```
C#  
public GameObject flame;
```

Stores reference to **flame object**.

Used to show/hide fire.

Example:

- Particle effect
 - Fire object
-

```
C#  
public GameObject smoke;
```

Stores **smoke object** reference.

Used to show smoke effect.

```
C#  
public Light candleLight;
```

Stores candle light reference.

Used for glow/illumination.

```
C#  
private bool isLit = false;
```

Boolean variable:

- **true** = candle ON
- **false** = candle OFF

Initially:

```
C#  
false
```

Means candle starts OFF.

4) Update Function

```
C#  
void Update()
```

Runs every frame continuously.

Checks mouse click.

5) Mouse Click Detection

```
C#
if (Mouse.current != null && Mouse.current.LeftButton.wasPressedThisFrame)
```

Meaning:

Mouse.current != null

Checks mouse is connected.

Safety check.

LeftButton.wasPressedThisFrame

Checks if left mouse clicked **once**.

Then run code.

6) Toggle Logic

```
C#
isLit = !isLit;
```

! means opposite.

So:

- false → true
- true → false

Example:

First click:

```
C#
false → true
```

Candle ON

Second click:

```
C#
true → false
```

Candle OFF

7) Flame ON/OFF

```
C#
flame.SetActive(isLit);
```

If **isLit = true**

→ flame visible

If **false**
→ flame hidden

8) Smoke ON/OFF

C#
smoke.SetActive(isLit);

If ON:
→ smoke visible

If OFF:
→ smoke hidden

9) Light ON/OFF

C#
candleLight.enabled = isLit;

If ON:
→ light enabled

If OFF:
→ light disabled

Makes realistic candle glow.

Full Flow

```

Game starts
↓
Candle OFF
↓
Mouse click
↓
isLit = true
↓
Flame ON
Smoke ON
Light ON
↓
Mouse click again
↓
isLit = false
↓
Everything OFF
  
```

Viva Answer (short)

“This script creates an interactive candle in Unity. On left mouse click, it toggles a boolean variable and activates/deactivates flame, smoke, and candle light, creating a candle ON/OFF effect.”



```

☐
using UnityEngine;
using UnityEngine.InputSystem;

public class EmotionChange : MonoBehaviour
{
    public GameObject neutralMouth;
    public GameObject happyMouth;
    public GameObject sadMouth;
    public GameObject angryMouth;
    public GameObject leftBrow;
    public GameObject rightBrow;

    int emotion = 0;

    void Update()
    {
        // New Input System mouse click
        if (Mouse.current != null && Mouse.current.leftButton.wasPressedThisFrame)
        {
            emotion++;

            if (emotion > 3)
                emotion = 0;

            ShowEmotion();
        }
    }

    void ShowEmotion()
    {
        neutralMouth.SetActive(false);
        happyMouth.SetActive(false);
        sadMouth.SetActive(false);
        angryMouth.SetActive(false);
        leftBrow.SetActive(false);
        rightBrow.SetActive(false);

        if (emotion == 0)
    
```

```

{
    neutralMouth.SetActive(true);
}
else if (emotion == 1)
{
    happyMouth.SetActive(true);
}
else if (emotion == 2)
{
    sadMouth.SetActive(true);
}
else if (emotion == 3)
{
    angryMouth.SetActive(true);
    leftBrow.SetActive(true);
    rightBrow.SetActive(true);
}
}
}

```

Script Name: EmotionChange

Purpose:

This script **changes a character's emotion on mouse click.**

It cycles through:

1. Neutral 😐
2. Happy 😊
3. Sad 😞
4. Angry 😡

Then repeats.

1) Libraries

C#
using UnityEngine;
using UnityEngine.InputSystem;

UnityEngine

Used for:

- GameObject
- MonoBehaviour

- SetActive()

UnityEngine.InputSystem

Used for **mouse click input** (New Input System).

2) Class

C#
public class *EmotionChange* : *MonoBehaviour*

Creates script named **EmotionChange**.

Attach it to character GameObject.

3) Variables

C#
public *GameObject* *neutralMouth*;
public *GameObject* *happyMouth*;
public *GameObject* *sadMouth*;
public *GameObject* *angryMouth*;

These store different **mouth shapes**.

Used for different emotions:

- Neutral mouth → 😐
- Happy mouth → 😊
- Sad mouth → 😞
- Angry mouth → 😡

Only one mouth is shown at a time.

C#
public *GameObject* *leftBrow*;
public *GameObject* *rightBrow*;

Stores eyebrow objects.

Used only for **angry expression** to make face look angry.

4) Emotion Variable

C#
int *emotion* = 0;

Stores current emotion number:

- 0 → Neutral
- 1 → Happy
- 2 → Sad
- 3 → Angry

Initially:

```
C#  
0
```

So character starts Neutral.

5) Update Function

```
C#  
void Update()
```

Runs every frame.

Checks mouse click.

6) Mouse Click Detection

```
C#  
if (Mouse.current != null && Mouse.current.LeftButton.wasPressedThisFrame)
```

Means:

If left mouse clicked once → change emotion.

7) Increase Emotion

```
C#  
emotion++;
```

Emotion number increases by 1.

Example:

0 → 1 → 2 → 3

8) Reset to Start

```
C#  
if (emotion > 3)  
    emotion = 0;
```

After angry (3), it goes back to neutral (0).

Cycle becomes:

Neutral → Happy → Sad → Angry → Neutral

9) Call Function

C#
`ShowEmotion();`

Updates character face.

10) ShowEmotion() Function

C#
`void ShowEmotion()`

Main function that shows correct expression.

First Hide Everything

C#
`neutralMouth.SetActive(false);
happyMouth.SetActive(false);
sadMouth.SetActive(false);
angryMouth.SetActive(false);
leftBrow.SetActive(false);
rightBrow.SetActive(false);`

Turns OFF all mouths and eyebrows.

Why?

Because only one emotion should be visible.

Neutral

C#
`if (emotion == 0)
{
 neutralMouth.SetActive(true);
}`

Shows neutral mouth.

Face = 😐

Happy

C#

```
else if (emotion == 1)
{
    happyMouth.SetActive(true);
}
```

Shows happy mouth.

Face = 😊

Sad

```
C#
else if (emotion == 2)
{
    sadMouth.SetActive(true);
}
```

Shows sad mouth.

Face = 😞

Angry

```
C#
else if (emotion == 3)
{
    angryMouth.SetActive(true);
    leftBrow.SetActive(true);
    rightBrow.SetActive(true);
}
```

Shows:

- angry mouth
- left eyebrow
- right eyebrow

Makes angry face = 😡

Full Flow

Start → Neutral 😐

Click → Happy 😊

Click → Sad 😞

Click → Angry 😡

Click → Neutral 😐

Repeat...

Viva Answer (short)

“This script changes a character’s facial emotion on mouse click. It uses an integer variable to track emotion state and activates the corresponding mouth and eyebrow GameObjects to show Neutral, Happy, Sad, and Angry expressions.”



```

☐
using UnityEngine;
using UnityEngine.InputSystem;

public class KickBall : MonoBehaviour
{
    Rigidbody rb;

    int speedLevel = 0;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
    }

    void Update()
    {
        // New Input System mouse click
        if (Mouse.current != null && Mouse.current.leftButton.wasPressedThisFrame)
        {
            rb.linearVelocity = Vector3.zero;
            rb.angularVelocity = Vector3.zero;

            transform.position = new Vector3(0, 0.5f, 0);

            speedLevel++;

            if (speedLevel > 3)
                speedLevel = 1;

            float force = 0;

            if (speedLevel == 1)
                force = 200f;    // slow
            else if (speedLevel == 2)
                force = 400f;    // medium
            else if (speedLevel == 3)
                force = 700f;    // fast

            rb.AddForce(Vector3.forward * force);
        }
    }
}

```

```
}  
}
```

Script Name: KickBall

Purpose:

This script **kicks a ball forward when mouse is clicked.**

Each click changes kick speed:

1. Slow kick 
2. Medium kick  
3. Fast kick   

Then repeats.

1) Libraries

C#
`using UnityEngine;`
`using UnityEngine.InputSystem;`

UnityEngine

Used for:

- Rigidbody
- Vector3
- Transform
- Physics

UnityEngine.InputSystem

Used for **mouse click input.**

2) Class

C#
`public class KickBall : MonoBehaviour`

Creates script named **KickBall.**

Attach it to ball GameObject.

3) Rigidbody Variable

C#
Rigidbody rb;

Stores ball's **Rigidbody component**.

Why?

Rigidbody allows:

- movement
- force
- gravity
- physics

Without Rigidbody, ball won't move physically.

4) Speed Variable

C#
int speedLevel = 0;

Stores current kick level:

- 1 = Slow
- 2 = Medium
- 3 = Fast

Starts from:

C#
0

5) Start Function

C#
void Start()
{
rb = GetComponent<Rigidbody>();
}

Runs once when game starts.

Gets Rigidbody attached to same object.

Example:

Ball GameObject → Rigidbody → stored in **rb**

Now script can control ball physics.

6) Update Function

C#
`void Update()`

Runs every frame.

Checks mouse click.

7) Mouse Click

C#
`if (Mouse.current != null && Mouse.current.leftButton.wasPressedThisFrame)`

If left mouse clicked once → kick ball.

8) Stop Old Movement

C#
`rb.linearVelocity = Vector3.zero;`
`rb.angularVelocity = Vector3.zero;`

Stops previous movement.

`linearVelocity = 0`

Stops moving direction.

`angularVelocity = 0`

Stops spinning/rotation.

Why?

Without this, old speed keeps affecting new kick.

9) Reset Position

C#
`transform.position = new Vector3(0, 0.5f, 0);`

Moves ball back to starting point.

Position:

- X = 0
- Y = 0.5
- Z = 0

So every kick starts from same place.

10) Increase Speed Level

C#
`speedLevel++;`

Each click increases level.

Example:

0 → 1 → 2 → 3

11) Reset Cycle

C#
`if (speedLevel > 3)
 speedLevel = 1;`

After level 3, back to 1.

Cycle:

Slow → Medium → Fast → Slow

12) Force Variable

C#
`float force = 0;`

Stores kick power.

13) Select Force

Slow

C#
`if (speedLevel == 1)
 force = 200f;`

Small force → slow kick.

Medium

C#
`else if (speedLevel == 2)
 force = 400f;`

Medium force → medium kick.

Fast

```
C#
else if (speedLevel == 3)
    force = 700f;
```

Large force → fast kick.

14) Kick Ball

```
C#
rb.AddForce(Vector3.forward * force);
```

Applies force in **forward direction (Z-axis)**.

Ball moves ahead.

Example:

- 200 → slow
- 400 → medium
- 700 → fast

Full Flow

Game starts

↓

Click 1 → Slow kick

↓

Click 2 → Medium kick

↓

Click 3 → Fast kick

↓

Click 4 → Slow kick again

↓

Repeat

Important Concept

GetComponent<Rigidbody>()

Gets Rigidbody from object.

Vector3.forward

Direction = forward.

AddForce()

Pushes object physically.

transform.position

Changes object location.

Viva Answer (short)

“This script creates a ball kicking system in Unity. On mouse click, it resets the ball position, increases speed level, selects force value, and applies forward force using Rigidbody physics to simulate slow, medium, and fast kicks.”



```

☐
using UnityEngine;

public class BounceControl : MonoBehaviour
{
    public float stopAfterSeconds = 10f;

    Rigidbody rb;
    float timer;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
        timer = stopAfterSeconds;
    }

    void Update()
    {
        timer -= Time.deltaTime;

        // gradually reduce bounce
        rb.linearVelocity = new Vector3(
            rb.linearVelocity.x * 0.995f,
            rb.linearVelocity.y,
            rb.linearVelocity.z * 0.995f
        );

        // stop after specified time
        if (timer <= 0)
    
```

```
{
    rb.linearVelocity = Vector3.zero;
    rb.angularVelocity = Vector3.zero;
    rb.useGravity = false;
    enabled = false;
}
}
```

Script Name: BounceControl

Purpose:

This script **gradually slows a bouncing ball/object** and **completely stops it after 10 seconds**.

So:

- Object bounces/moves normally first 🏀
- Slowly loses speed
- After 10 sec → fully stops

1) Library

C#
using *UnityEngine;*

Used for:

- Rigidbody
- Time
- Vector3
- MonoBehaviour

(No Input System here because no mouse/keyboard input is used.)

2) Class

C#
public class *BounceControl* : *MonoBehaviour*

Creates script named **BounceControl**.

Attach it to bouncing object.

Example:

- Ball
 - Cube
 - Sphere
-

3) Public Variable

C#
`public float stopAfterSeconds = 10f;`

Time after which object stops.

Default:

C#
`10 seconds`

Can change in Inspector.

Example:

- 5 → stop in 5 sec
 - 20 → stop in 20 sec
-

4) Rigidbody Variable

C#
`Rigidbody rb;`

Stores Rigidbody reference.

Used to control:

- speed
 - rotation
 - gravity
-

5) Timer Variable

C#
`float timer;`

Countdown timer.

Tracks remaining time.

Starts at 10 → goes down → 0

6) Start Function

C#
`void Start()`

Runs once when game starts.

Get Rigidbody

C#
`rb = GetComponent<Rigidbody>();`

Gets Rigidbody component from object.

Now script can control physics.

Set Timer

C#
`timer = stopAfterSeconds;`

Copies stop time into timer.

Example:

C#
`timer = 10`

Countdown starts.

7) Update Function

C#
`void Update()`

Runs every frame.

Continuously updates timer and speed.

8) Reduce Timer

C#
`timer -= Time.deltaTime;`

Subtracts frame time.

Time.deltaTime

Time taken for one frame (~0.016 sec at 60 FPS)

Example:

10 → 9.98 → 9.96 → ... → 0

This creates real-time countdown.

9) Gradually Slow Movement

C#
`rb.linearVelocity = new Vector3(`

Changes object speed.

X speed

C#
`rb.linearVelocity.x * 0.995f`

Reduces X speed slightly.

Example:

100 → 99.5 → 99 → 98.5

Slowly slows down.

Y speed

C#
`rb.linearVelocity.y`

Keeps vertical speed same.

Why?

To allow natural bouncing.

Z speed

C#
`rb.linearVelocity.z * 0.995f`

Also reduces forward/backward speed slowly.

Meaning

Every frame:

- X slows
- Z slows
- Y stays normal

This creates **friction/damping effect**.

10) Stop Object

```
C#  
if (timer <= 0)
```

When timer reaches 0 → stop everything.

Stop Movement

```
C#  
rb.linearVelocity = Vector3.zero;
```

Velocity becomes 0.

Object stops moving.

Stop Rotation

```
C#  
rb.angularVelocity = Vector3.zero;
```

Stops spinning.

Disable Gravity

```
C#  
rb.useGravity = false;
```

Gravity OFF.

Object stays still.

Disable Script

```
C#  
enabled = false;
```

Stops Update() from running.

Saves performance.

Script becomes inactive.

Full Flow

Object starts moving

↓

Timer = 10 sec

↓

Every frame speed reduces a little

↓
 Movement slows gradually
 ↓
 Timer reaches 0
 ↓
 Velocity = 0
 Rotation = 0
 Gravity OFF
 Script OFF
 ↓
 Object fully stops

Important Concepts

Time.deltaTime

Frame-independent timing.

Vector3.zero

Means:

C#
(0,0,0)

No movement.

enabled = false

Disables script.

Viva Answer (short)

“This script controls bouncing motion by gradually reducing Rigidbody velocity every frame, creating a damping effect. A timer counts down, and after the specified time the object’s movement, rotation, and gravity are disabled so it completely stops.”



```

☐
using UnityEngine;
using UnityEngine.InputSystem;

public class LampToggle : MonoBehaviour
{
    public Light lampLight;
  
```

```
private bool isOn = true;

void Update()
{
    if (Mouse.current.leftButton.wasPressedThisFrame)
    {
        isOn = !isOn;
        lampLight.enabled = isOn;
    }
}
}
```

Script Name: LampToggle

Purpose:

This script **turns a lamp/light ON and OFF on mouse click.**

Click once → Light OFF 💡❌

Click again → Light ON 💡✅

Keeps toggling.

1) Libraries

C#
using *UnityEngine;*
using *UnityEngine.InputSystem;*

UnityEngine

Used for:

- Light
- MonoBehaviour

UnityEngine.InputSystem

Used for **mouse click detection** (New Input System).

2) Class

C#
public class *LampToggle : MonoBehaviour*

Creates script named **LampToggle**.

Attach this script to lamp GameObject.

3) Light Variable

```
C#  
public Light lampLight;
```

Stores reference to the lamp's **Light component**.

Why?

Because script needs control of:

- ON state
- OFF state

In Inspector, drag your lamp light here.

4) Boolean Variable

```
C#  
private bool isOn = true;
```

Stores lamp state.

- `true` = ON
- `false` = OFF

Initially:

```
C#  
true
```

Lamp starts ON.

5) Update Function

```
C#  
void Update()
```

Runs every frame.

Checks for mouse click continuously.

6) Mouse Click Check

```
C#  
if (Mouse.current.LeftButton.wasPressedThisFrame)
```

Meaning:

If left mouse button is clicked once → run code.

(`wasPressedThisFrame` means only one-time click detection.)

7) Toggle State

C#
`isOn = !isOn;`

! means opposite value.

So:

- true → false
- false → true

Example:

1st click:

C#
`true → false`

Lamp OFF

2nd click:

C#
`false → true`

Lamp ON

8) Enable/Disable Light

C#
`lampLight.enabled = isOn;`

Controls light component.

If:

C#
`isOn = true`

→ Light ON

If:

C#
`isOn = false`

→ Light OFF

Full Flow

Game starts → Lamp ON

↓

Mouse click

↓

`isOn = false`

↓

Light OFF

↓

Mouse click again

↓

`isOn = true`

↓

Light ON

↓

Repeat

Important Concepts

bool

Stores:

- true
- false

!

Reverse/toggle operator.

Light.enabled

Turns light component ON/OFF.

Viva Answer (short)

“This script creates a lamp toggle system in Unity. On left mouse click, it reverses a boolean variable and enables or disables the Light component, making the lamp turn ON and OFF.”



```
using UnityEngine;
using UnityEngine.InputSystem;

public class CharacterInteract : MonoBehaviour
{
    int state = 0;
    Vector3 startPos;

    void Start()
    {
        startPos = transform.position;
    }

    void Update()
    {
        // Left click → cycle actions
        if (Mouse.current != null && Mouse.current.leftButton.wasPressedThisFrame)
        {
            state++;

            if (state > 3)
                state = 1;

            if (state == 1)
            {
                // move
                transform.position += new Vector3(1f, 0, 0);
            }
            else if (state == 2)
            {
                // rotate
                transform.Rotate(0, 90, 0);
            }
            else if (state == 3)
            {
                // jump
                transform.position = startPos + new Vector3(0, 1f, 0);
            }
        }

        // Right click → reset
        if (Mouse.current != null && Mouse.current.rightButton.wasPressedThisFrame)
        {
            transform.position = startPos;
            transform.rotation = Quaternion.identity;
        }
    }
}
```

Script Name: CharacterInteract

Purpose:

This script makes a character perform actions on mouse clicks.

Left click → changes action:

1. Move → character moves right 🚶
2. Rotate → character turns ↻
3. Jump → character moves up ⬆

Right click → Reset character ↺

1) Libraries

```
C#
using UnityEngine;
using UnityEngine.InputSystem;
```

UnityEngine

Used for:

- Vector3
- Transform
- Quaternion
- MonoBehaviour

UnityEngine.InputSystem

Used for mouse input.

2) Class

```
C#
public class CharacterInteract : MonoBehaviour
```

Creates script named **CharacterInteract**.

Attach it to character GameObject.

3) Variables

State Variable

```
C#
int state = 0;
```

Stores current action.

- 1 = Move
- 2 = Rotate
- 3 = Jump

Starts at:

```
C#
0
```

Start Position

```
C#
Vector3 startPos;
```

Stores original character position.

Used for reset.

4) Start Function

```
C#
void Start()
{
    startPos = transform.position;
}
```

Runs once when game starts.

Stores current position in **startPos**.

Example:

If character starts at:

```
C#
(0,0,0)
```

Then:

```
C#
startPos = (0,0,0)
```

Saved for later reset.

5) Update Function

```
C#
void Update()
```

Runs every frame.

Checks left and right mouse clicks.

LEFT CLICK ACTIONS

6) Detect Left Click

```
C#
if (Mouse.current != null && Mouse.current.LeftButton.wasPressedThisFrame)
```

If left click → perform next action.

7) Increase State

```
C#
state++;
```

Example:

0 → 1 → 2 → 3

8) Reset Cycle

```
C#
if (state > 3)
    state = 1;
```

After 3, go back to 1.

Cycle:

Move → Rotate → Jump → Move

9) Move Action

```
C#
transform.position += new Vector3(1f, 0, 0);
```

Moves character on X-axis.

Meaning:

- +1 right
- no Y change
- no Z change

Example:

C#
 $(0, 0, 0) \rightarrow (1, 0, 0)$

Character moves right.

10) Rotate Action

C#
`transform.Rotate(0, 90, 0);`

Rotates character:

- X = 0
- Y = 90°
- Z = 0

Turns character sideways.

11) Jump Action

C#
`transform.position = startPos + new Vector3(0, 1f, 0);`

Moves character upward.

Example:

Start:

C#
 $(0, 0, 0)$

Jump:

C#
 $(0, 1, 0)$

Looks like jump.

RIGHT CLICK RESET

12) Detect Right Click

C#
`if (Mouse.current != null && Mouse.current.rightButton.wasPressedThisFrame)`

If right click → reset.

13) Reset Position

C#

```
transform.position = startPos;
```

Moves character back to original position.

14) Reset Rotation

C#

```
transform.rotation = Quaternion.identity;
```

Resets rotation to default.

Quaternion.identity means:

0 rotation

Character faces original direction.

Full Flow

Game starts

↓

Position saved

↓

Left Click 1 → Move 

Left Click 2 → Rotate 

Left Click 3 → Jump 

Left Click 4 → Move again

Right Click → Reset 

Important Concepts

transform.position

Changes object position.

transform.Rotate()

Rotates object.

Quaternion.identity

Default rotation.

Vector3

Stores X, Y, Z position.

Viva Answer (short)

“This script creates character interaction in Unity. Left mouse click cycles through move, rotate, and jump actions using a state variable. Right mouse click resets the character’s position and rotation to its original state.”



```

☐
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerController : MonoBehaviour
{
    public float moveSpeed = 5f;
    public Transform groundCheck; // 🔥 Empty GameObject under player
    public LayerMask groundLayer; // 🔥 Set to Ground layer
    public float groundCheckRadius = 0.2f; // 🔥 How big the check circle should be

    private Rigidbody2D rb;
    private PlayerInputActions playerInputActions;
    private Vector2 moveInput;
    private bool isGrounded; // 🔥 Whether the player is touching the ground

    private void Awake()
    {
        rb = GetComponent<Rigidbody2D>();
        playerInputActions = new PlayerInputActions();

        playerInputActions.Player.Enable();
        playerInputActions.Player.Move.performed += OnMovePerformed;
        playerInputActions.Player.Move.canceled += OnMoveCanceled;
    }

    private void OnMovePerformed(InputAction.CallbackContext context)
    {
        moveInput = context.ReadValue<Vector2>();
    }

    private void OnMoveCanceled(InputAction.CallbackContext context)
    {
        moveInput = Vector2.zero;
    }
}

```

```
private void Update()
{
    // 🔥 Ground Check Logic
    isGrounded = Physics2D.OverlapCircle(groundCheck.position, groundCheckRadius,
    groundLayer);

    // 🔥 Optional: Visualize ground check in Scene view
    Debug.DrawRay(groundCheck.position, Vector3.down * groundCheckRadius, Color.red);
}

private void FixedUpdate()
{
    rb.linearVelocity = moveInput * moveSpeed;
}

private void OnEnable()
{
    playerInputActions.Player.Enable();
}

private void OnDisable()
{
    playerInputActions.Player.Disable();
    playerInputActions.Player.Move.performed -= OnMovePerformed;
    playerInputActions.Player.Move.canceled -= OnMoveCanceled;
}
}
```

Script Name: PlayerController

Purpose:

This script controls a **2D player movement** using Unity's **New Input System**.

Player can:

- Move left 
- Move right 
- Move up/down (if input given)
- Detect whether touching ground 

(Mainly used for 2D character controller.)

1) Libraries

C#

```
using UnityEngine;  
using UnityEngine.InputSystem;
```

UnityEngine

Used for:

- Rigidbody2D
- Transform
- Physics2D
- LayerMask

UnityEngine.InputSystem

Used for New Input System controls.

2) Class

```
C#  
public class PlayerController : MonoBehaviour
```

Creates script named **PlayerController**.

Attach to Player GameObject.

3) Public Variables

Move Speed

```
C#  
public float moveSpeed = 5f;
```

Controls player speed.

Example:

- 2 = slow
 - 5 = normal
 - 10 = fast
-

Ground Check

```
C#  
public Transform groundCheck;
```

Reference point under player.

Usually an **empty GameObject** placed below feet.

Used to check:

Is player standing on ground?

Ground Layer

```
C#  
public LayerMask groundLayer;
```

Specifies which objects count as ground.

Example:

- Floor
- Platform

Unity checks only this layer.

Ground Check Radius

```
C#  
public float groundCheckRadius = 0.2f;
```

Size of checking circle.

Small circle below feet.

If circle touches ground → grounded.

4) Private Variables

Rigidbody2D

```
C#  
private Rigidbody2D rb;
```

Controls 2D physics movement.

Needed for moving player.

Input Actions

```
C#  
private PlayerInputActions playerInputActions;
```

Stores generated Input Action class.

Used for keyboard/controller input.

Example:
WASD / Arrow keys.

Move Input

C#
`private Vector2 moveInput;`

Stores movement direction.

Example:

Right:

C#
`(1,0)`

Left:

C#
`(-1,0)`

Up:

C#
`(0,1)`

No input:

C#
`(0,0)`

Grounded Boolean

C#
`private bool isGrounded;`

Stores:

- true = on ground
- false = in air

5) Awake()

C#
`private void Awake()`

Runs before Start().

Used for setup.

Get Rigidbody

```
C#  
rb = GetComponent<Rigidbody2D>();
```

Gets Rigidbody2D attached to player.

Now script can move player.

Create Input System

```
C#  
playerInputActions = new PlayerInputActions();
```

Creates input actions object.

Enable Controls

```
C#  
playerInputActions.Player.Enable();
```

Turns input ON.

Subscribe Move Event

```
C#  
playerInputActions.Player.Move.performed += OnMovePerformed;
```

When movement key pressed:

run **OnMovePerformed()**.

Subscribe Cancel Event

```
C#  
playerInputActions.Player.Move.canceled += OnMoveCanceled;
```

When key released:

run **OnMoveCanceled()**.

6) OnMovePerformed()

```
C#  
private void OnMovePerformed(InputAction.CallbackContext context)
```

Runs when movement key is pressed.

```
C#
moveInput = context.ReadValue<Vector2>();
```

Reads input direction.

Example:

D key →

```
C#
(1,0)
```

Stored in `moveInput`.

7) OnMoveCanceled()

```
C#
moveInput = Vector2.zero;
```

When key released:

Movement becomes:

```
C#
(0,0)
```

Player stops.

8) Update()

```
C#
private void Update()
```

Runs every frame.

Used for ground checking.

Ground Check

```
C#
isGrounded = Physics2D.OverlapCircle(...)
```

Creates invisible circle below player.

Checks if touching ground layer.

If yes:

```
C#
true
```

Else:

```
C#
false
```


Debug Line

C#
Debug.DrawRay(...)

Draws red line in Scene View.

Used only for debugging.

Helps see check area.

9) FixedUpdate()

C#
private void FixedUpdate()

Runs on physics timing.

Best place for Rigidbody movement.

C#
*rb.linearVelocity = moveInput * moveSpeed;*

Moves player.

Example:

Input:

C#
(1,0)

Speed:

C#
5

Velocity:

C#
(5,0)

Player moves right.

10) OnEnable()

C#
playerInputActions.Player.Enable();

Re-enables controls when object becomes active.

11) OnDisable()

C#

```
playerInputActions.Player.Disable();
```

Disables controls when object is inactive.

Also removes events:

```
C#
-= OnMovePerformed
-= OnMoveCanceled
```

Why?

To avoid memory leaks / duplicate events.

Full Flow

Game starts

↓

Input system enabled

↓

Player presses key

↓

`moveInput` gets direction

↓

`FixedUpdate` moves player

↓

Ground check checks feet touching ground

↓

Key released

↓

Movement stops

Important Note

`isGrounded` is currently **only checked**, not used.

Normally it can be used for:

- Jump only when grounded
 - Landing effects
 - Fall detection
-

Viva Answer (short)

“This script creates a 2D player controller using Unity’s New Input System. It reads movement input, moves the player using `Rigidbody2D` velocity, and uses `Physics2D` `OverlapCircle` below the

player to detect whether the player is on the ground.”



```

☐
using UnityEngine;
using UnityEngine.InputSystem;

public class PlayerMove : MonoBehaviour
{
    public float speed = 5f;

    Rigidbody rb;
    Vector2 moveInput;

    void Awake()
    {
        rb = GetComponent<Rigidbody>();
    }

    void Update()
    {
        // New Input System keyboard/gamepad movement
        if (Keyboard.current != null)
        {
            Vector2 input = Vector2.zero;

            if (Keyboard.current.aKey.isPressed)
                input.x -= 1;
            if (Keyboard.current.dKey.isPressed)
                input.x += 1;
            if (Keyboard.current.wKey.isPressed)
                input.y += 1;
            if (Keyboard.current.sKey.isPressed)
                input.y -= 1;

            moveInput = input.normalized;
        }

        Vector3 move = new Vector3(moveInput.x, 0, moveInput.y) * speed;

        rb.linearVelocity = new Vector3(
            move.x,
            rb.linearVelocity.y,
            move.z
        );
    }

    void OnCollisionEnter(Collision collision)

```

```
{
    Debug.Log("Collided with: " + collision.gameObject.name);
}
```

Script Name: PlayerMove

Purpose:

This script moves a **3D player** using **WASD keys** with Unity's **New Input System**.

Controls:

- **W** → Forward 
- **S** → Backward 
- **A** → Left 
- **D** → Right 

It also detects collision with objects.

1) Libraries

```
C#
using UnityEngine;
using UnityEngine.InputSystem;
```

UnityEngine

Used for:

- Rigidbody
- Vector2 / Vector3
- Collision
- Debug

UnityEngine.InputSystem

Used for keyboard input.

2) Class

```
C#
public class PlayerMove : MonoBehaviour
```

Creates script named **PlayerMove**.

Attach to Player GameObject.

3) Speed Variable

C#
public float speed = 5f;

Controls movement speed.

Example:

- 2 = slow
- 5 = normal
- 10 = fast

Can change in Inspector.

4) Rigidbody Variable

C#
Rigidbody rb;

Stores Rigidbody component.

Needed for:

- physics movement
 - gravity
 - collision
-

5) Input Variable

C#
Vector2 moveInput;

Stores movement direction.

Example:

Right:

C#
(1,0)

Left:

C#
(-1,0)

Forward:

C#
(0,1)

Backward:

```
C#
(0, -1)
```

6) Awake()

```
C#
void Awake()
{
    rb = GetComponent<Rigidbody>();
}
```

Runs at start.

Gets Rigidbody attached to player.

Now script can move object.

7) Update()

```
C#
void Update()
```

Runs every frame.

Reads keyboard input + moves player.

Check Keyboard

```
C#
if (Keyboard.current != null)
```

Checks keyboard exists.

Safety check.

Create Input Variable

```
C#
Vector2 input = Vector2.zero;
```

Starts with no movement:

```
C#
(0, 0)
```

A Key

```
C#
if (Keyboard.current.aKey.isPressed)
    input.x -= 1;
```

Move left.

X becomes negative.

D Key

```
C#
if (Keyboard.current.dKey.isPressed)
    input.x += 1;
```

Move right.

X becomes positive.

W Key

```
C#
if (Keyboard.current.wKey.isPressed)
    input.y += 1;
```

Move forward.

Y becomes positive.

S Key

```
C#
if (Keyboard.current.sKey.isPressed)
    input.y -= 1;
```

Move backward.

Y becomes negative.

Normalize Input

```
C#
moveInput = input.normalized;
```

Makes movement length = 1.

Why?

Prevents diagonal movement from becoming faster.

Example:

W + D:

Without normalize → faster diagonal

With normalize → same speed in all directions 

8) Convert to 3D Movement

```
C#
Vector3 move = new Vector3(moveInput.x, 0, moveInput.y) * speed;
```

Converts 2D input into 3D movement.

Notice:

- X → left/right
- Y → set to 0
- input.y → becomes Z (forward/back)

Example:

W key:

```
C#
(0,0,1)
```

Forward movement.

9) Apply Velocity

```
C#
rb.linearVelocity = new Vector3(
    move.x,
    rb.linearVelocity.y,
    move.z
);
```

Sets Rigidbody speed.

Important:

Keeps:

```
C#
rb.linearVelocity.y
```

same.

Why?

So gravity/jumping still works.

Only changes:

- X movement
 - Z movement
-

10) Collision Function

```
C#
void OnCollisionEnter(Collision collision)
```


Runs when player hits something.

Example:

- wall
- cube
- obstacle

Print Object Name

C#

```
Debug.Log("Collided with: " + collision.gameObject.name);
```

Prints in Console:

Example:

C#

```
Collided with: Wall
```

Useful for debugging.

Full Flow

Game starts

↓

Get Rigidbody

↓

Press WASD

↓

Read input

↓

Normalize direction

↓

Convert to 3D movement

↓

Apply velocity

↓

Player moves

↓

Hit object → collision message printed

Important Concepts

normalized

Keeps same speed in all directions.

linearVelocity

Controls movement speed.

OnCollisionEnter

Detects collision.

Debug.Log

Prints message in Console.

Viva Answer (short)

“This script creates 3D player movement using WASD keys. It reads keyboard input with Unity’s New Input System, normalizes movement direction, converts it to 3D velocity, and applies it through Rigidbody. It also detects collisions and prints the collided object’s name in the Console.”



```

☐
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.XR.ARFoundation;
using UnityEngine.XR.ARSubsystems;

public class ImageTracker : MonoBehaviour
{
    public ARTrackedImageManager trackedImageManager;
    public GameObject carPrefab;

    private Dictionary<string, GameObject> spawnedObjects = new Dictionary<string, GameObject>
    ();

    void OnEnable()
    {
        trackedImageManager.trackedImagesChanged += OnImageChanged;
    }

    void OnDisable()
    {
        trackedImageManager.trackedImagesChanged -= OnImageChanged;
    }

    void OnImageChanged(ARTrackedImagesChangedEventArgs eventArgs)
    
```

```
{
    foreach (var image in eventArgs.added)
    {
        SpawnOrUpdate(image);
    }

    foreach (var image in eventArgs.updated)
    {
        SpawnOrUpdate(image);
    }
}

void SpawnOrUpdate(ARTrackedImage image)
{
    string name = image.referenceImage.name;

    if (!spawnedObjects.ContainsKey(name))
    {
        GameObject obj = Instantiate(carPrefab, image.transform);
        spawnedObjects[name] = obj;
    }

    spawnedObjects[name].SetActive(image.trackingState == TrackingState.Tracking);

    spawnedObjects[name].transform.position = image.transform.position;
    spawnedObjects[name].transform.rotation = image.transform.rotation;
}
}
```

Script Name: ImageTracker

Purpose:

This script is for **AR (Augmented Reality) Image Tracking**.

When camera detects a marker/image:

→ it **spawns a 3D car model** 🚗

If marker moves:

→ car moves with it

If marker disappears:

→ car hides

This is used in **marker-based AR**.

1) Libraries

C#

```
using System.Collections.Generic;  
using UnityEngine;  
using UnityEngine.XR.ARFoundation;  
using UnityEngine.XR.ARSubsystems;
```

System.Collections.Generic

Used for:

C#
Dictionary

Dictionary stores spawned objects.

UnityEngine

Basic Unity features:

- GameObject
 - Transform
 - Instantiate
-

UnityEngine.XR.ARFoundation

Used for AR features like:

- image tracking
 - AR camera
 - AR managers
-

UnityEngine.XR.ARSubsystems

Used for:

- TrackingState
 - AR tracking data
-

2) Class

C#
public class *ImageTracker* : *MonoBehaviour*

Creates script named **ImageTracker**.

Attach to AR Session Origin / XR Origin object.

3) Variables

AR Image Manager

C#
public *ARTrackedImageManager* *trackedImageManager*;

Reference to Unity AR Image Tracking Manager.

Its job:

Detect reference images/markers.

Example:

- company logo
- printed card
- poster

When detected → event triggers.

3D Model

C#
public *GameObject* *carPrefab*;

Prefab to spawn.

Example:

 Car model

When marker found → spawn this.

Dictionary

C#
private *Dictionary*<*string*, *GameObject*> *spawnedObjects*

Stores spawned AR objects.

Format:

ImageName -> **SpawnedObject**

Example:

carMarker -> **CarObject**

Why?

Prevents spawning duplicates.

4) OnEnable()

C#

```
void OnEnable()
```

Runs when script becomes active.

```
C#
```

```
trackedImageManager.trackedImagesChanged += OnImageChanged;
```

Subscribes to AR tracking event.

Meaning:

If image:

- detected
- updated
- removed

Run:

```
C#
```

```
OnImageChanged()
```

5) OnDisable()

```
C#
```

```
void OnDisable()
```

Runs when script is disabled.

```
C#
```

```
trackedImageManager.trackedImagesChanged -= OnImageChanged;
```

Unsubscribes event.

Why?

Avoids:

- duplicate calls
 - memory leaks
-

6) Image Changed Function

```
C#
```

```
void OnImageChanged(ARTrackedImagesChangedEventArgs eventArgs)
```

Runs whenever tracked image changes.

Can detect:

- Added image
- Updated image
- Removed image

Added Images

```
C#
foreach (var image in eventArgs.added)
```

Loops through newly detected markers.

Example:

Camera sees marker first time.

Then:

```
C#
SpawnOrUpdate(image);
```

Spawn object.

Updated Images

```
C#
foreach (var image in eventArgs.updated)
```

When marker position changes.

Example:

Move printed marker paper.

Then AR object updates position.

Again calls:

```
C#
SpawnOrUpdate(image);
```

Update object.

7) SpawnOrUpdate()

```
C#
void SpawnOrUpdate(ARTrackedImage image)
```

Main function.

Handles:

- spawn
 - show/hide
 - move
 - rotate
-

Get Marker Name

```
C#
string name = image.referenceImage.name;
```

Gets tracked image name.

Example:

Marker name:

CarMarker

Stored in:

```
C#
name
```

Check Already Spawned

```
C#
if (!spawnedObjects.ContainsKey(name))
```

If object doesn't exist → create it.

If exists → don't create again.

Prevents duplicates.

Spawn Object

```
C#
GameObject obj = Instantiate(carPrefab, image.transform);
```

Creates car prefab.

Parent = tracked image.

Meaning:

Marker moves → car moves with marker.

Save in Dictionary

```
C#
spawnedObjects[name] = obj;
```

Stores object.

Example:

CarMarker -> **CarObject**

Now script remembers it.

8) Show/Hide Object

C#

```
spawnedObjects[name].SetActive(...)
```

Checks tracking state.

Condition:

C#

```
image.trackingState == TrackingState.Tracking
```

If marker visible:

→ object active

If marker lost:

→ object hidden

Very important for AR.

9) Update Position

C#

```
spawnedObjects[name].transform.position = image.transform.position;
```

Moves object to marker position.

Marker moves → car follows.

10) Update Rotation

C#

```
spawnedObjects[name].transform.rotation = image.transform.rotation;
```

Rotates object same as marker.

Rotate marker paper → car rotates too.

Full Flow

App starts

↓

Camera scans image marker

↓

ARFoundation detects marker

↓

OnImageChanged() called

↓

SpawnOrUpdate() called

↓

Car prefab spawned 🚗



Move marker → car moves



Rotate marker → car rotates



Marker lost → car hides

Important Concepts

ARTrackedImageManager

Detects reference images.

Instantiate()

Creates prefab.

Dictionary

Stores spawned objects.

TrackingState.Tracking

Checks if marker is visible.

SetActive()

Show/hide object.

Viva Answer (short)

“This script performs marker-based AR image tracking. When a reference image is detected, it spawns a 3D car prefab on that image, stores it in a Dictionary, updates its position and rotation as the marker moves, and hides it when tracking is lost.”



```

☐
using UnityEngine;
public class MaterialChanger : MonoBehaviour
{
    public Renderer cubeRenderer;
    public Renderer planeRenderer;
    public Renderer sphereRenderer;

```

```

public Material cubeMat1;
public Material cubeMat2;
public Material planeMat1;
public Material planeMat2;
public Material sphereMat1;
public Material sphereMat2;
bool changed = false;
public void ChangeObjects()
{
    changed = !changed;
    if (changed)
    {
        cubeRenderer.material = cubeMat2;
        planeRenderer.material = planeMat2;
        sphereRenderer.material = sphereMat2;
    }
    else
    {
        cubeRenderer.material = cubeMat1;
        planeRenderer.material = planeMat1;
        sphereRenderer.material = sphereMat1;
    }
}

```

Script Name: MaterialChanger

Purpose:

This script **changes the material/texture of Cube, Plane, and Sphere** when a button/function is called.

Example:

First click:

- Cube → Material 2
- Plane → Material 2
- Sphere → Material 2

Second click:

- All return to Material 1

So it **toggles between two sets of materials**.

1) Library

C#
using *UnityEngine*;

Used for:

- Renderer
- Material
- MonoBehaviour

No Input System here because this script is usually called by a **UI Button click**.

2) Class

C#
`public class MaterialChanger : MonoBehaviour`

Creates script named **MaterialChanger**.

Attach it to any GameObject (like Empty Object / Manager object).

3) Renderer Variables

C#
`public Renderer cubeRenderer;
public Renderer planeRenderer;
public Renderer sphereRenderer;`

These store Renderer components of:

- Cube
- Plane
- Sphere

What is Renderer?

Renderer displays object's:

- material
- color
- texture

Without Renderer, script can't change appearance.

In Inspector:

Drag:

- Cube → cubeRenderer
 - Plane → planeRenderer
 - Sphere → sphereRenderer
-

4) Material Variables

Cube Materials

```
C#
public Material cubeMat1;
public Material cubeMat2;
```

Two materials for cube.

Example:

- cubeMat1 = Red
 - cubeMat2 = Blue
-

Plane Materials

```
C#
public Material planeMat1;
public Material planeMat2;
```

Two materials for plane.

Example:

- Grass texture
 - Sand texture
-

Sphere Materials

```
C#
public Material sphereMat1;
public Material sphereMat2;
```

Two materials for sphere.

Example:

- Yellow
 - Green
-

5) Boolean Variable

```
C#
bool changed = false;
```

Tracks current state.

- **false** → Material Set 1

- `true` → Material Set 2

Initially:

C#
`false`

Objects start with first materials.

6) Main Function

C#
`public void ChangeObjects()`

Main function that changes materials.

Usually connected to:

UI Button OnClick()

When button pressed → this runs.

7) Toggle State

C#
`changed = !changed;`

Opposite value.

So:

false → true

true → false

This creates toggle effect.

8) If Changed = True

C#
`if (changed)`

If true → apply Material Set 2.

Cube

C#
`cubeRenderer.material = cubeMat2;`

Cube gets second material.

Plane

```
C#  
planeRenderer.material = planeMat2;
```

Plane gets second material.

Sphere

```
C#  
sphereRenderer.material = sphereMat2;
```

Sphere gets second material.

9) Else

```
C#  
else
```

If false → apply Material Set 1.

Cube

```
C#  
cubeRenderer.material = cubeMat1;
```

Cube back to first material.

Plane

```
C#  
planeRenderer.material = planeMat1;
```

Plane back to first material.

Sphere

```
C#  
sphereRenderer.material = sphereMat1;
```

Sphere back to first material.

Full Flow

Start:

- Cube → Material 1
- Plane → Material 1
- Sphere → Material 1

Button click:

↓

changed = true

↓

All change to Material 2

Button click again:

↓

changed = false

↓

All return to Material 1

Repeat.

Important Concepts

Renderer

Controls visible appearance.

Material

Defines:

- color
- texture
- shader

.material

Used to assign new material.

bool

Tracks toggle state.

Viva Answer (short)

“This script changes the materials of cube, plane, and sphere. It stores two materials for each object and uses a boolean variable to toggle between material set 1 and material set 2 whenever

the `ChangeObjects()` function is called.”